Word2Vec

Department of Computer Science,
National Tsing Hua University, Taiwan

Deep Learning

Word embedding

- How to represent words in machine learning?

My favorite pet is cat .

  - Treat words as discrete atomic symbols, and therefore ‘cat’ may be represented as ‘1’ and ‘pet’ as ‘2.’

  - However, this embedding includes no semantics. Tasks learning from this representation causes models hard to train.

  - Better solutions?

Word embedding

- Intuition: Modeling relationships between words.
  - Semantics Similarity
    - CBOW (Continuous Bag-of-Words model),
    - Skip-Gram
  - Grammar / Syntax

CBOW & Skip-Gram

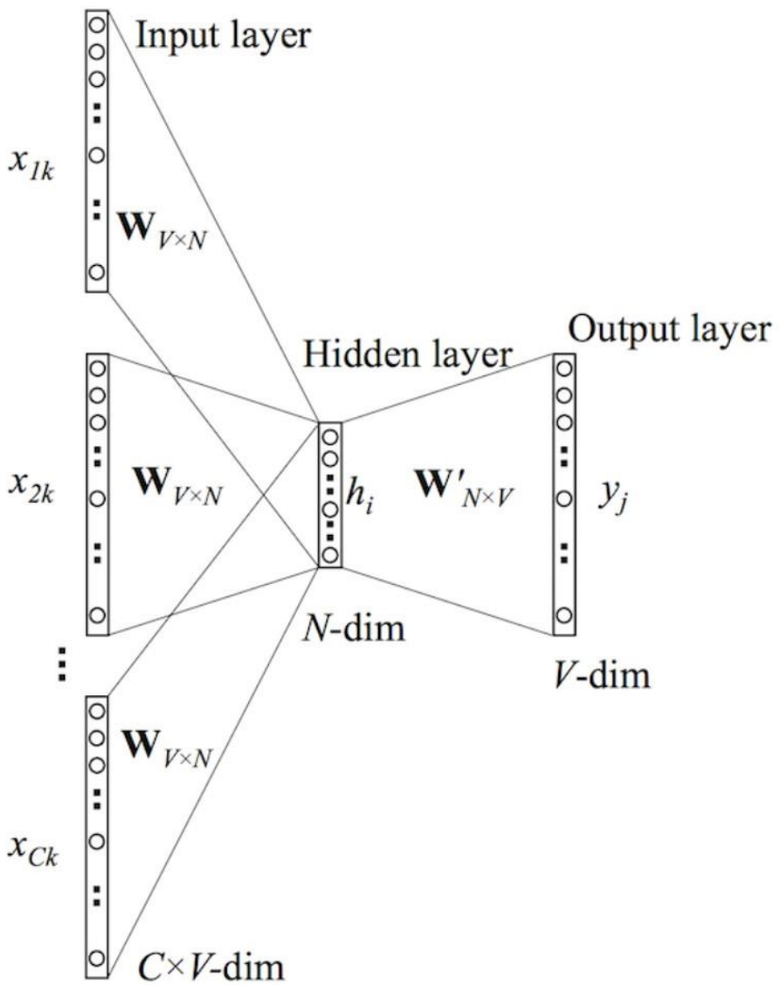
- Assumptions in CBOW & Skip-Gram:

  - Words with similar meaning / domain have the close contexts.

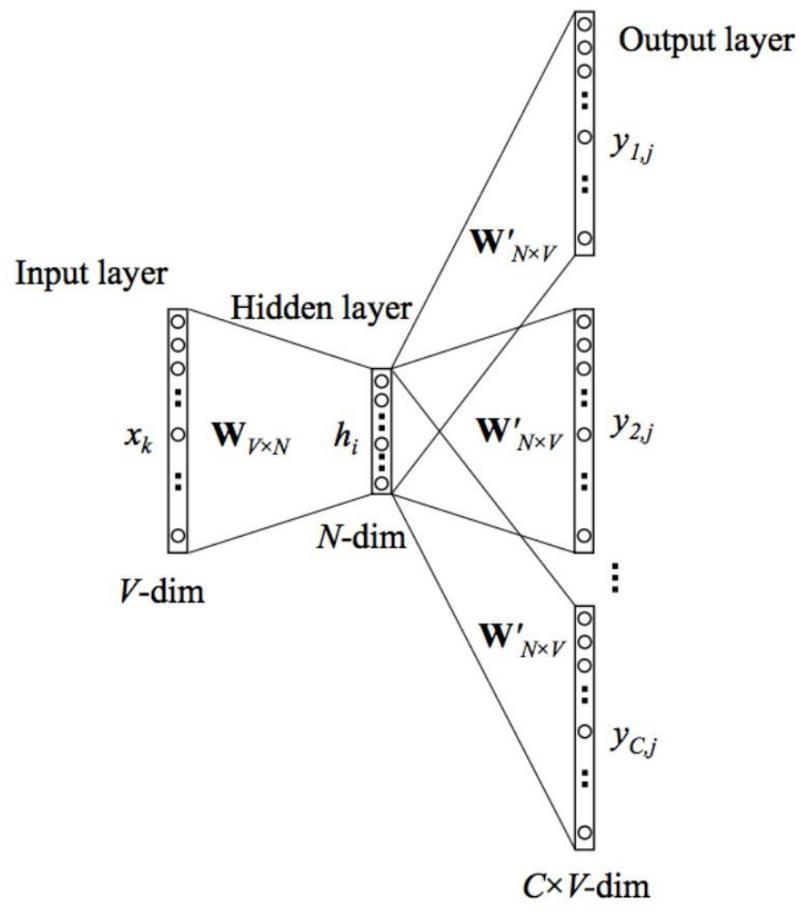
My favorite pet is ___ . *cat*, *dog*

CBOW & Skip-Gram

- CBOW: Predicting the target words with context words.



- Skip-Gram: Predicting context words from the target words.



CBOW & Skip-Gram

- For example, given the sentence:

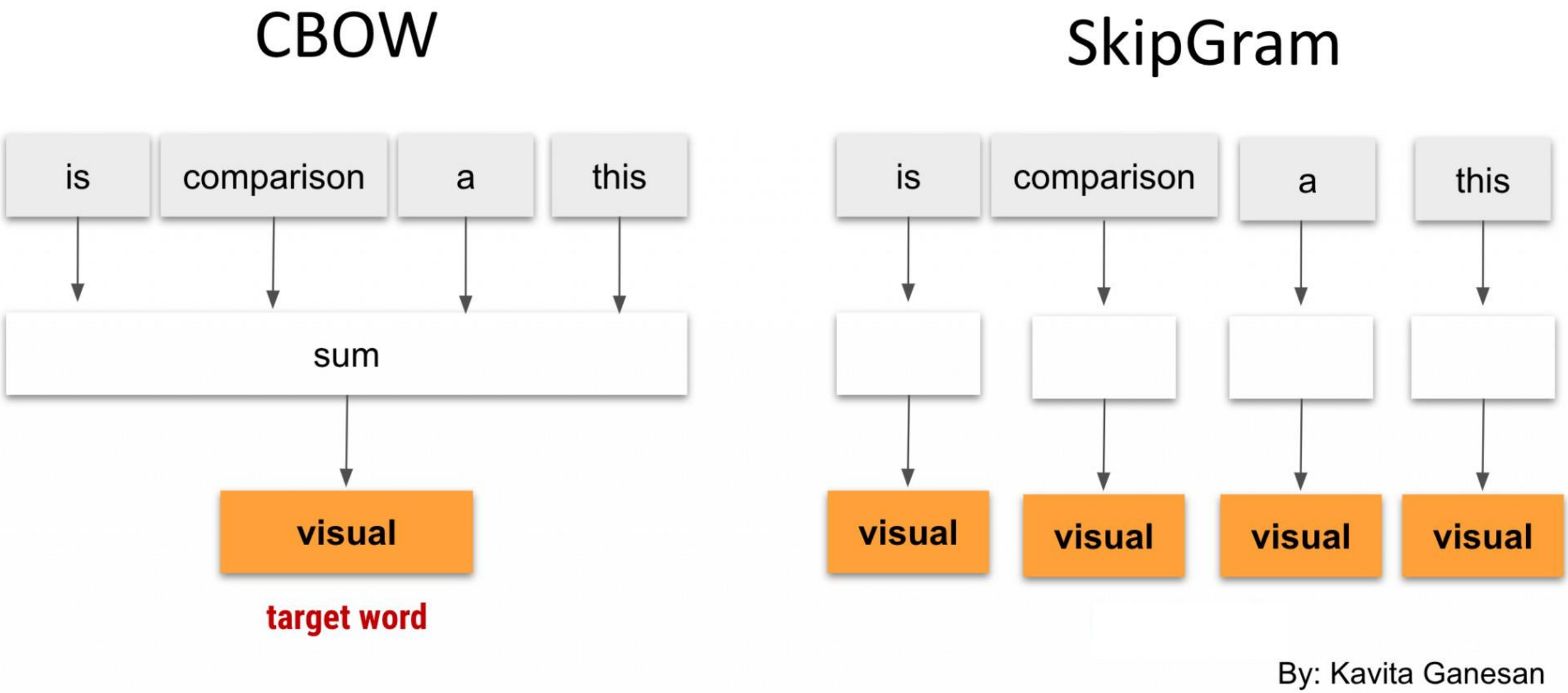
  the quick brown fox jumped over the lazy dog.

- CBOW will be trained on the dataset:

  ```
  ([the, brown], quick), ([quick, fox], brown), …
  ```

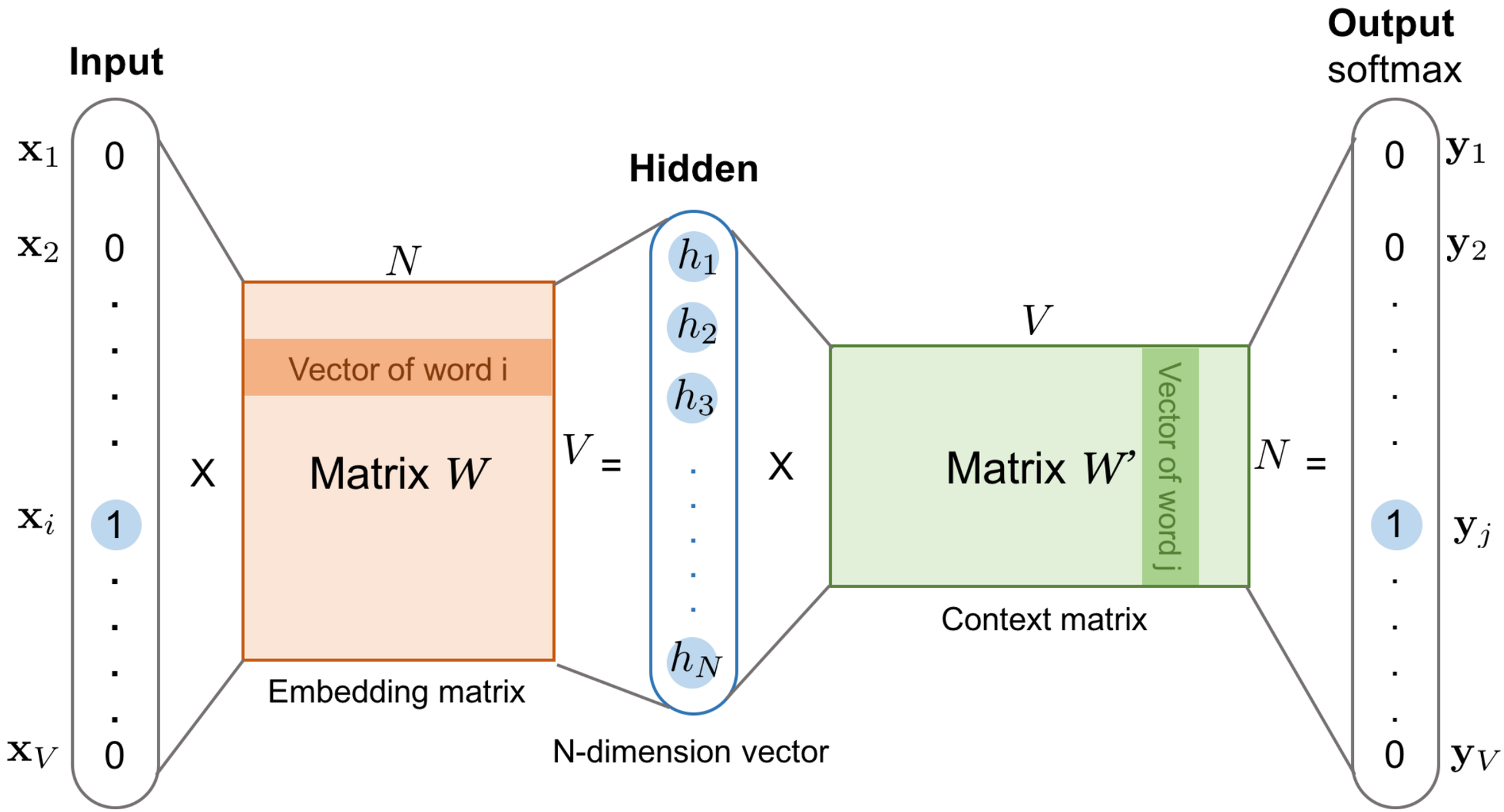
- Skip-Gram will be trained on the dataset:

  ```
  (quick, the), (quick, brown), (brown, quick),
  (brown, fox)…
  ```



Picture from https://kavita-ganesan.com/

Skip-Gram



Picture from https://lilianweng.github.io/

Softmax & Words classification

- Recap the softmax function:

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

- While training the embedding, we need to sum over the entire vocabulary in the denominator, which is costly.

$$p(w = i \mid \mathbf{x}) = \frac{\exp(h^\top v_{w_i'})}{\sum_{j \in V} \exp(h^\top v_{w_j'})}$$

$|V|$ **(vocabulary size) is thousands of millions**

Noise Contrastive Estimation

- Intuition: Estimate the parameters by learning to discriminate between the data and some artificially generated noise $\tilde{}$.

maximum likelihood estimation → noise contrastive estimation

- Objective function:

$$\arg\max \log p(d = 1 \mid w_t, x) + \sum_{\tilde{w}_i \sim Q}^{N} \log p(d = 0 \mid \tilde{w}_i, x)$$

- Derive form:

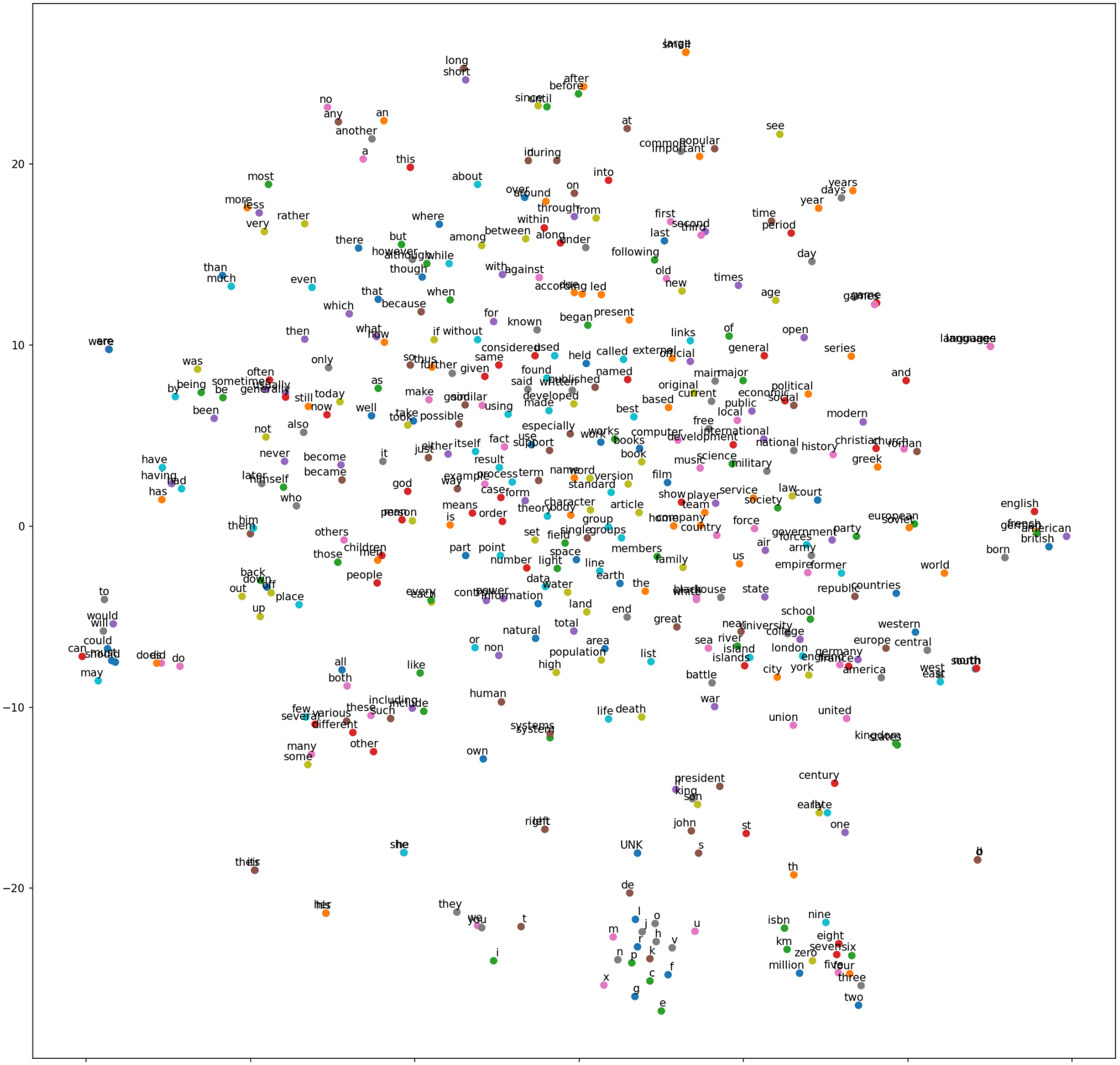
$$\arg\max \log \frac{p(w_t \mid x)}{p(w_t \mid x) + Nq(\tilde{w})} + \sum_{\tilde{w}_i \sim Q}^{N} \log \frac{Nq(\tilde{w}_i)}{p(w_t \mid x) + Nq(\tilde{w}_i)}$$

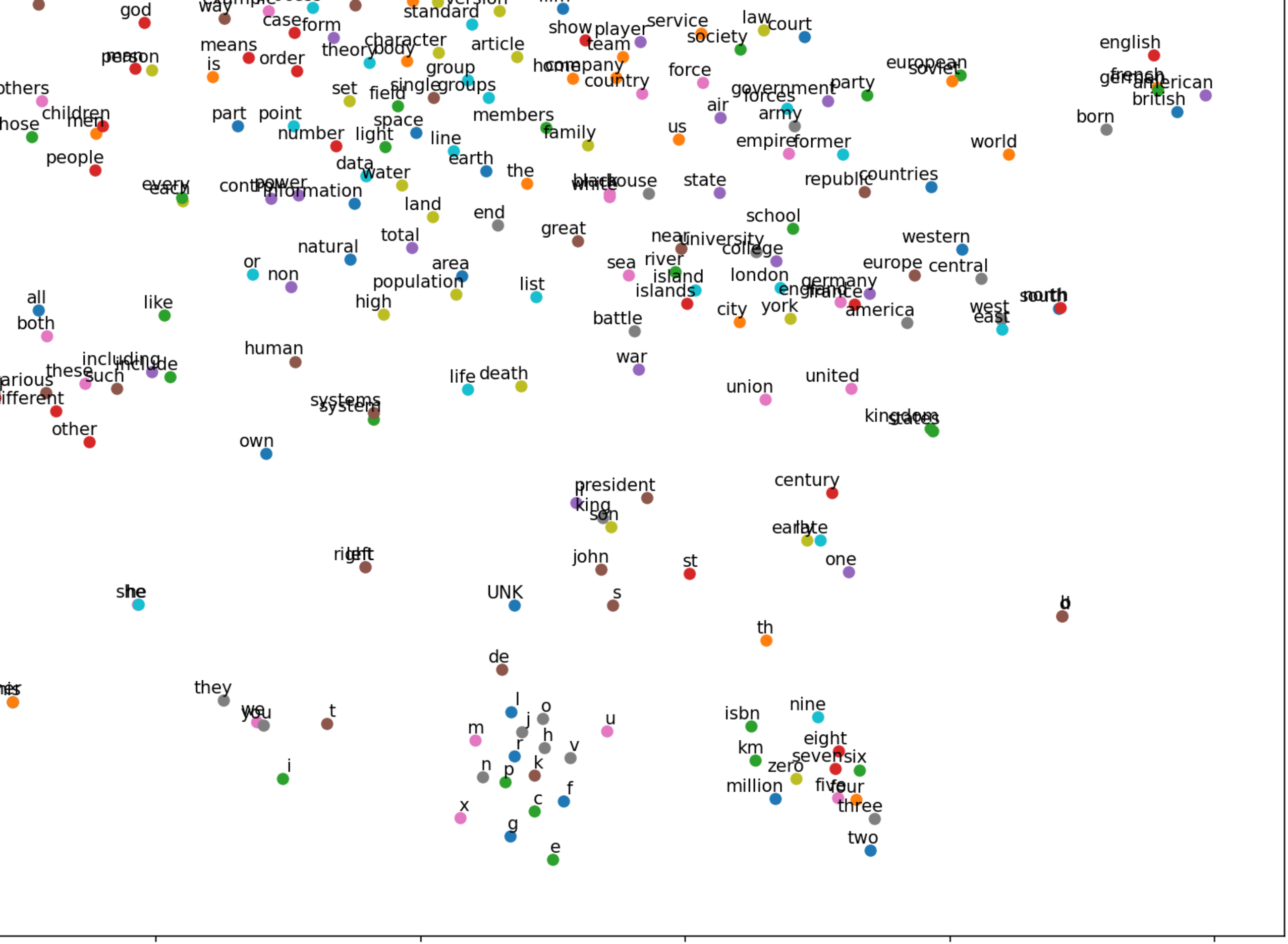
Noise Contrastive Estimation

- Intuition: Estimate the parameters by learning to discriminate between the data and some artificially generated noise ˜.

maximum likelihood estimation → noise contrastive estimation

- After training: evaluate with softmax function.


20
10
0
−10
−20
−30
−20
−10
0
10
20
30


god
way
case
form
standard
show
player
service
law
court
society
means
order
is
theory
body
character
article
team
company
country
force
european
soviet
english
french
german
american
british
born
others
children
men
hose
people
set
field
single
groups
group
part
point
space
members
family
us
air
army
government
party
forces
empire
former
world
number
light
line
earth
data
water
the
power
information
control
every
each
land
end
great
total
natural
or
non
area
population
high
list
sea
river
island
islands
london
england
france
germany
europe
central
western
school
university
college
near
state
republic
countries
white
house
black
all
both
like
city
york
america
west
east
south
battle
war
human
including
include
these
such
various
different
other
life
death
systems
system
own
union
united
kingdom
states
president
king
son
century
early
late
one
john
st
right
left
she
he
UNK
s
th
de
they
we
you
t
i
l
o
j
u
m
h
r
v
n
p
k
f
c
x
g
e
isbn
nine
eight
km
seven
six
zero
million
five
four
three
two
−10
0
10
20
30

Homework

- Model as a Function

```
embedding_layer = embedding_lookup().to(device)
nce_layer = nce_loss().to(device)

def word2vec(center_words, target_words):
    embedding = embedding_layer(center_words)
    loss = nce_layer((embedding, target_words))
    return loss

word2vec_parameters = list(embedding_layer.parameters()) + list(nce_layer.parameters())
```

[12]

Homework

- Model subclassing:

```
# embedding matrix - hidden layer
class embedding_lookup(nn.Module):
    def __init__(self):
        super(embedding_lookup, self).__init__()
        self.embedding_matrix = nn.Parameter(torch.empty(vocabulary_size, embed_size))
        nn.init.xavier_uniform_(self.embedding_matrix)    # Glorot uniform

    def forward(self, inputs):
        center_words = inputs
        embedding = F.embedding(center_words, self.embedding_matrix)
        return embedding
```

Reference

- https://ruder.io/word-embeddings-softmax/index.html#noisecontrastiveestimation
- https://lilianweng.github.io/lil-log/2017/10/15/learning-word-embedding.html
- https://aegis4048.github.io/optimize_computational_efficiency_of_skip-gram_with_negative_sampling
- http://proceedings.mlr.press/v9/gutmann10a/gutmann10a.pdf
- https://papers.nips.cc/paper/5165-learning-word-embeddings-efficiently-with-noise-contrastive-estimation.pdf
- https://docs.pytorch.org/tutorials/beginner/basics/buildmodel_tutorial.html